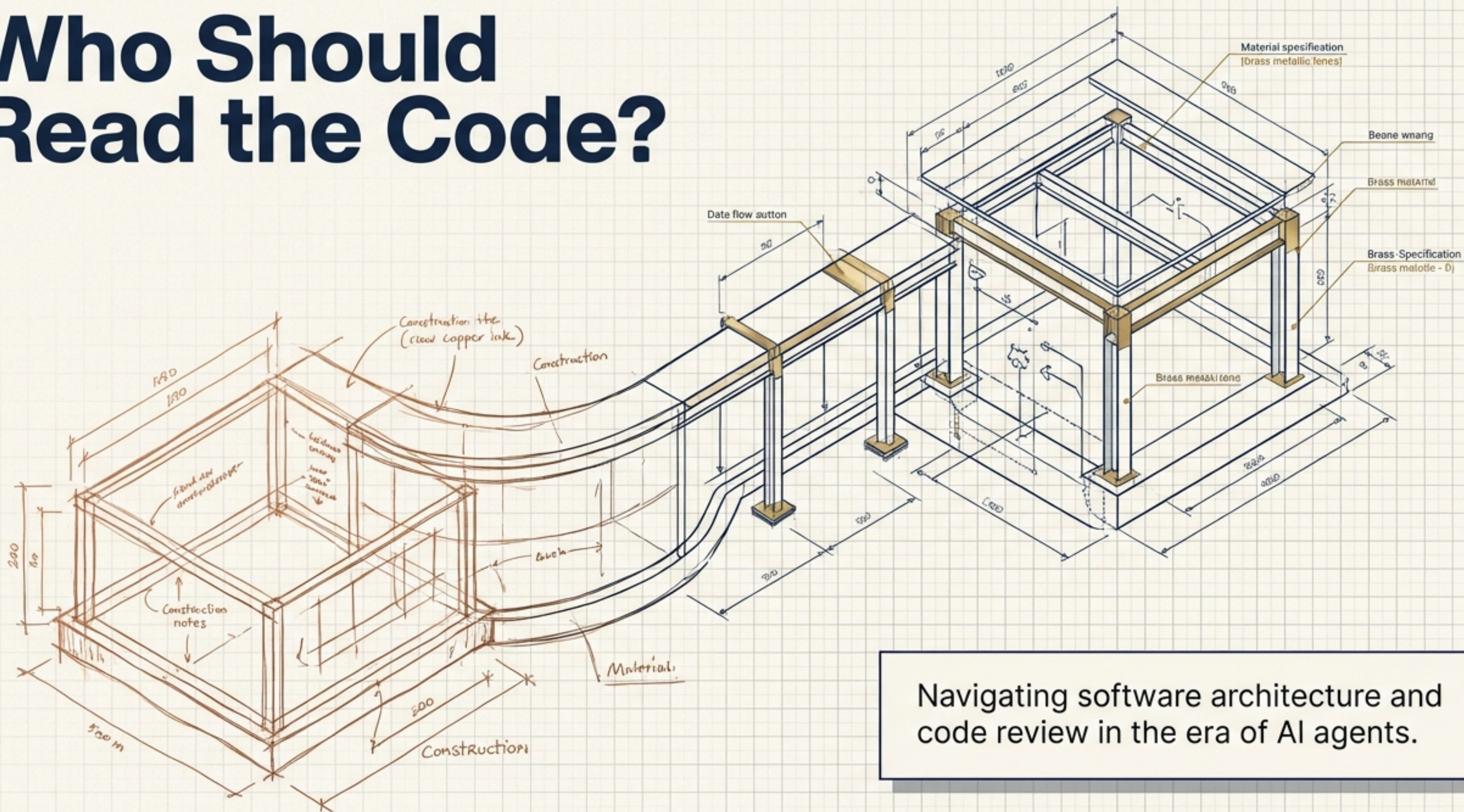
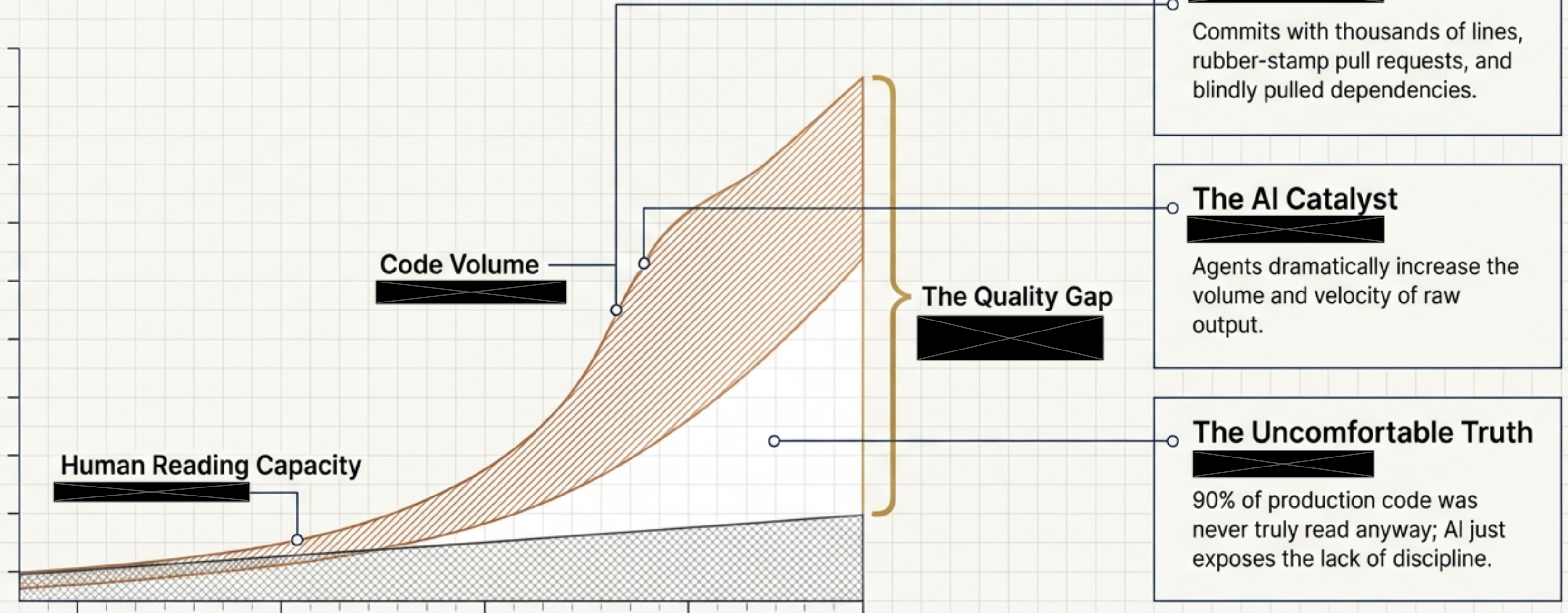


Who Should Read the Code?

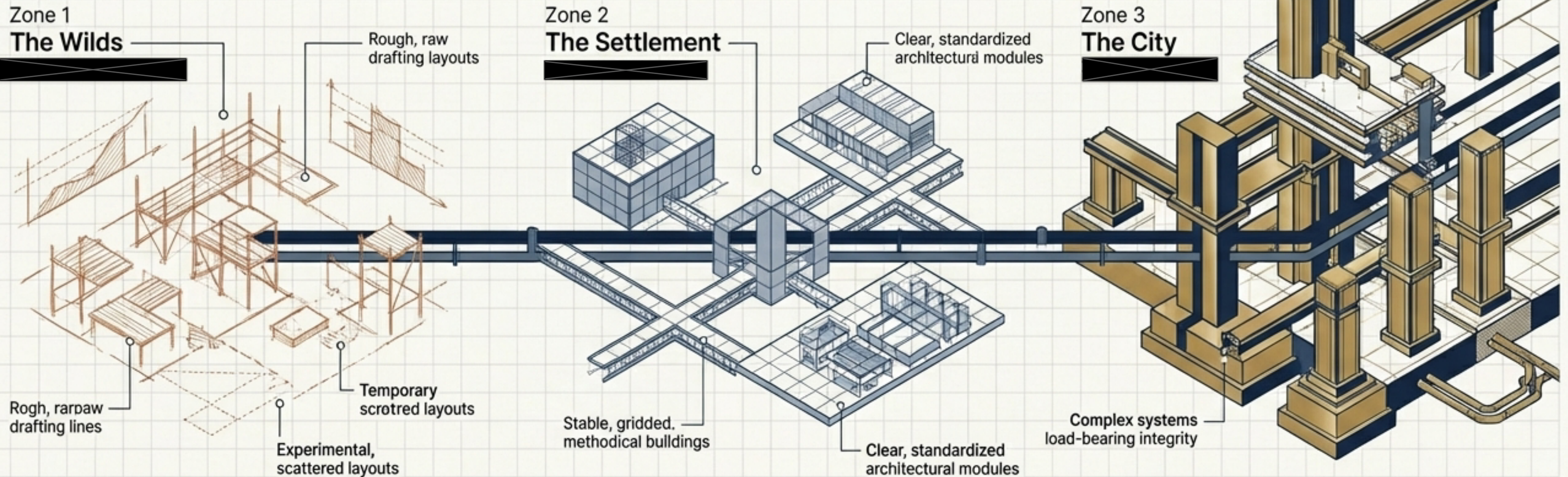


Navigating software architecture and code review in the era of AI agents.

The quality gap is an old problem, accelerated by AI.

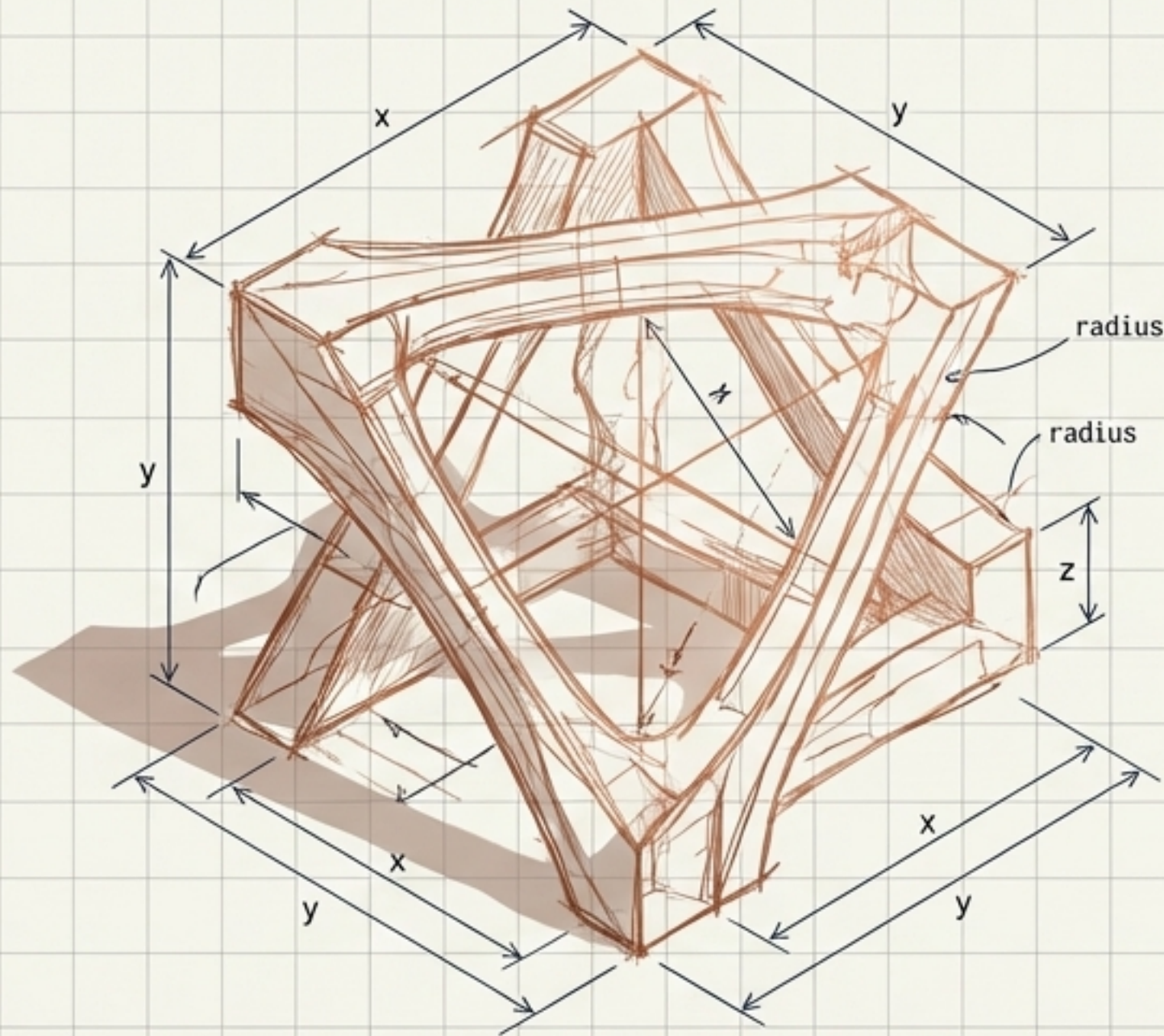


Reading code is now a phase-deppendent discipline.



The question isn't whether code should be read, but when, why, and by whom as it moves from experimentation to production.

The Explorer Phase: Code functions as a working brief.



The Goal

Functionality-first, engineering-second. Rapid prototyping and experimentation to learn exactly what works.

The Readers

Mostly AI agents running high-volume security scans, basic quality checks, and automated dependency audits.

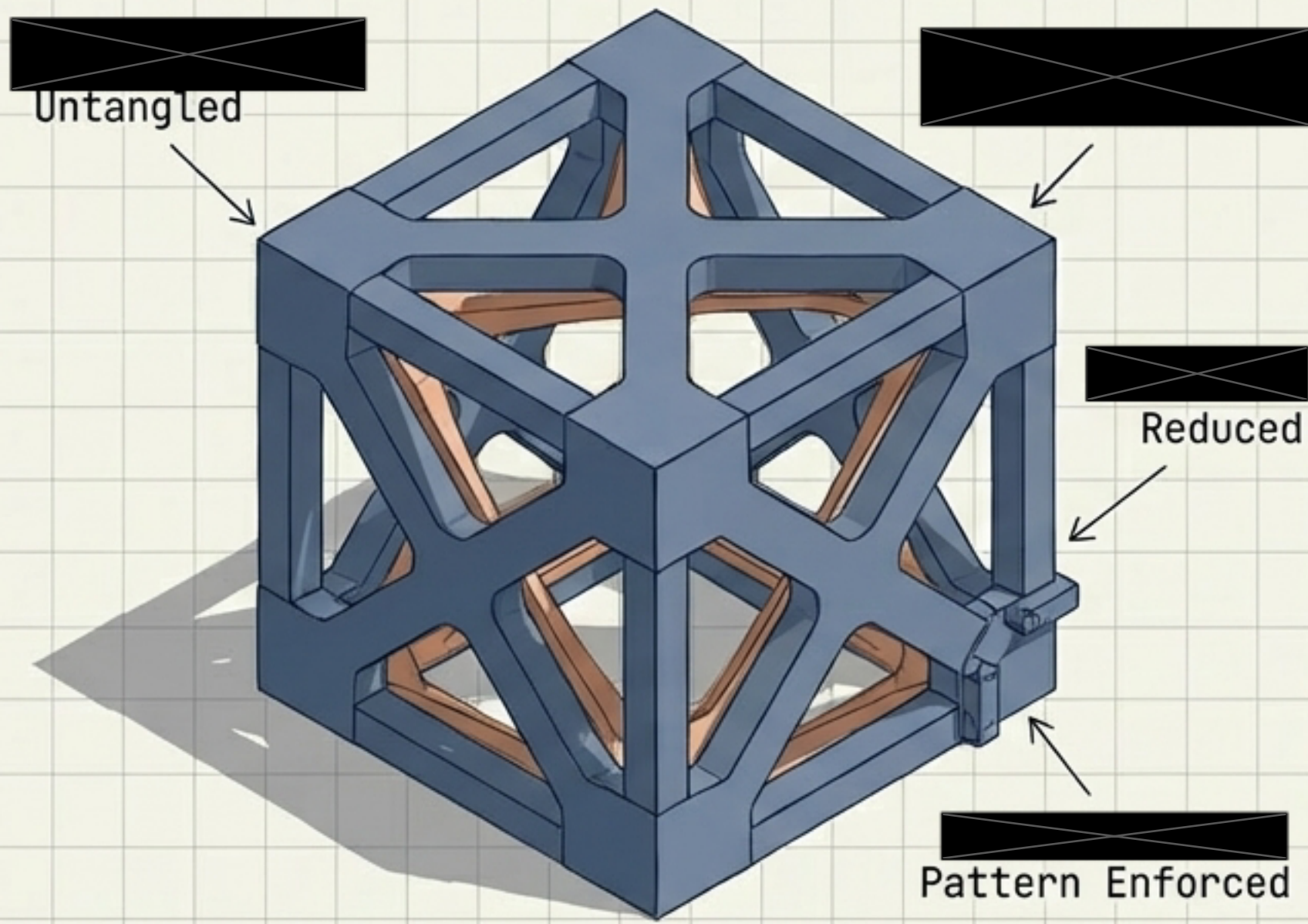
Human Involvement

Product owners or developers merely glance at the code to verify functionality. Intent matters significantly more than structure here.

Definition of Done

It works and we learned something valuable.

The Villager Phase: Engineering standards are enforced.



The Handover

This is where the Explorer's functional prototype is completely rebuilt, heavily refactored, and rigorously tested.

The Readers

Mid-level engineers building their architectural intuition. They read, question, and deeply understand the code.

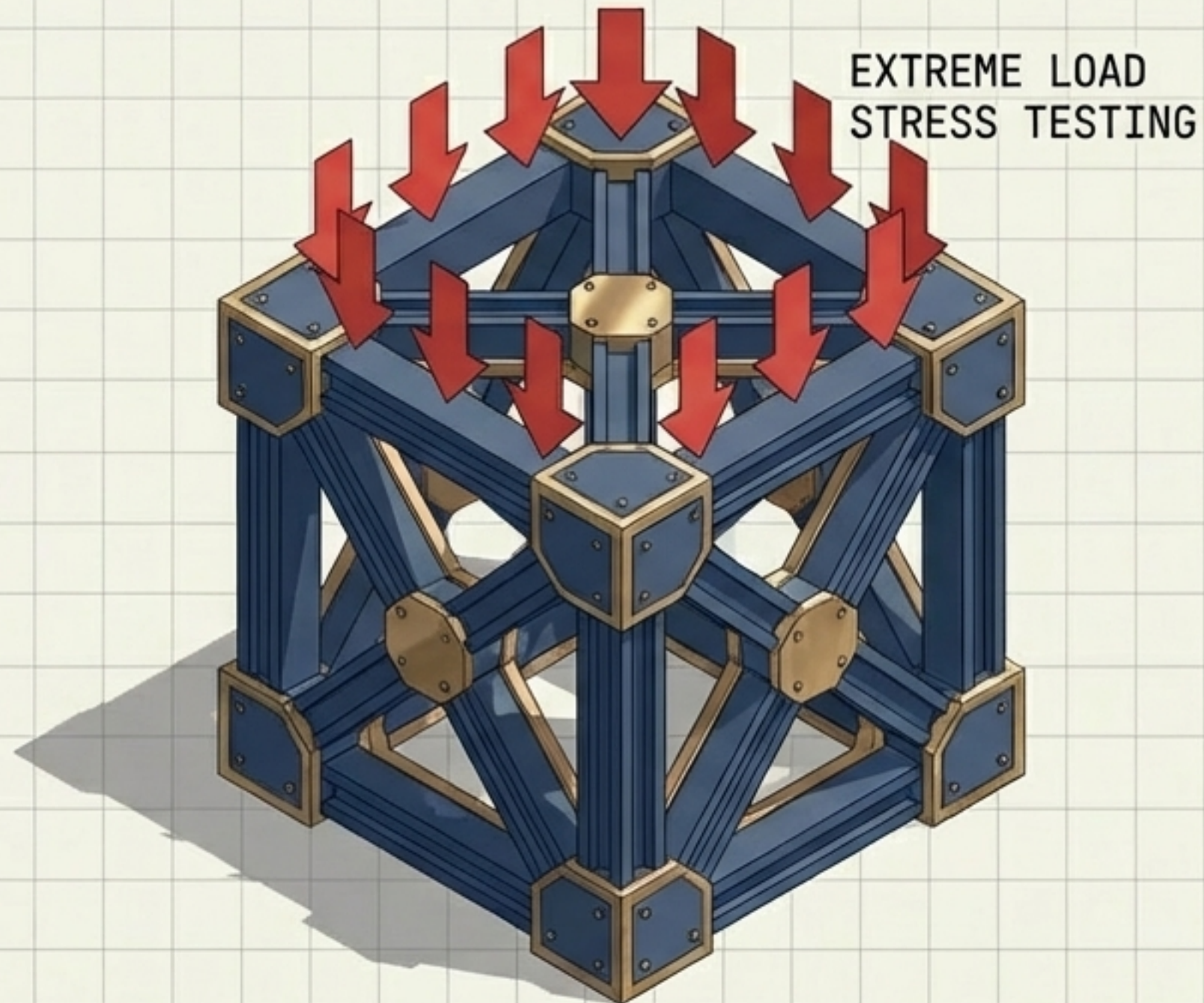
The Work

Architecture reviews, aggressive code reduction (removing Explorer duplication), pattern enforcement, and dependency untangling.

Definition of Done

Cleaner, smaller, well-structured, and fully covered by automated tests.

The Town Planner Phase: Code is scrutinised for survival.



The Readers

Senior developers and architects (15–30 years of experience). They read it with a pen, actively debating every design decision.

The Focus

Performance under extreme load, deep architectural security properties, and anticipating systemic edge cases.

The Standard

What happens when this code runs continuously for two years and the original authors have left the company?

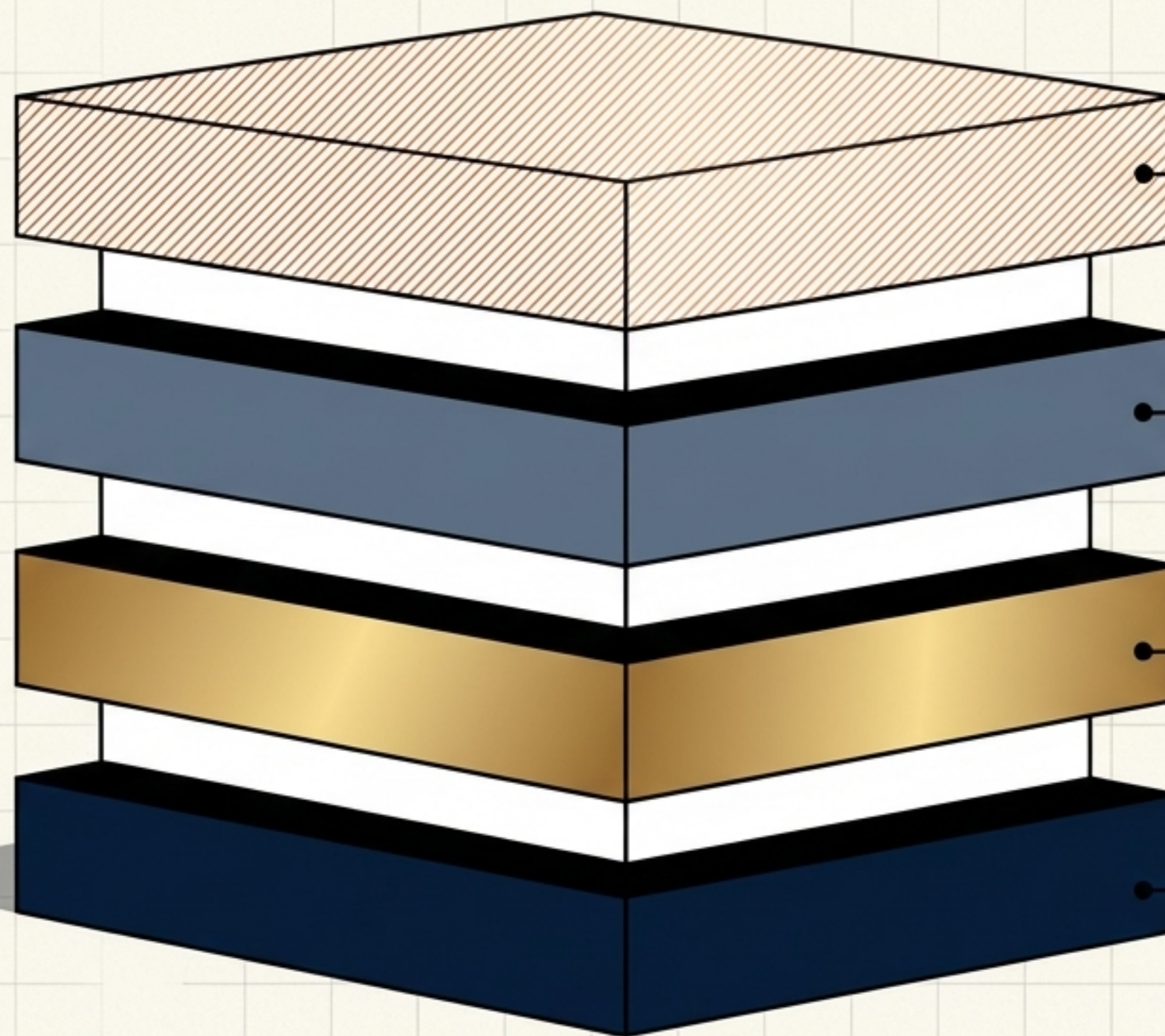
Definition of Done

I would bet the company on this code running reliably.

The AI-Era Review Framework

	The Explorer	The Villager	The Town Planner
Goal	Learn what works	Build to standards	Ensure system survival
Primary Reader	Agents (Humans glance)	Mid-level developers	Senior Architects
Reading Intent	Verify functionality	Rebuild & Refactor	Verify edge cases & scale
Definition of Done	'It works and I learned'	Cleaner & Tested	'Bet the company'
Environment	Experimental	Dev / Staging	Pre-Prod / Prod-path

Environments must strictly mirror the phases.



Explorer Environment

Unstable, throw-away, experimental.
Prevents AI hallucinations from escaping.

Villager Environment

Dev, staging, tested. Possesses its own
isolated CI pipeline and rigorous quality gates.

Town Planner Environment

Production-path, pre-prod, heavily
load-tested, structurally hardened.

Production Environment

Production is Production. Zero
experimental interference.

The Feedback Loop: Code flows forward, guidance flows backward.



Villager to Explorer

Villagers find errors and write explicit guidance to continually update Explorer prompts, role definitions, and AI agent skills.

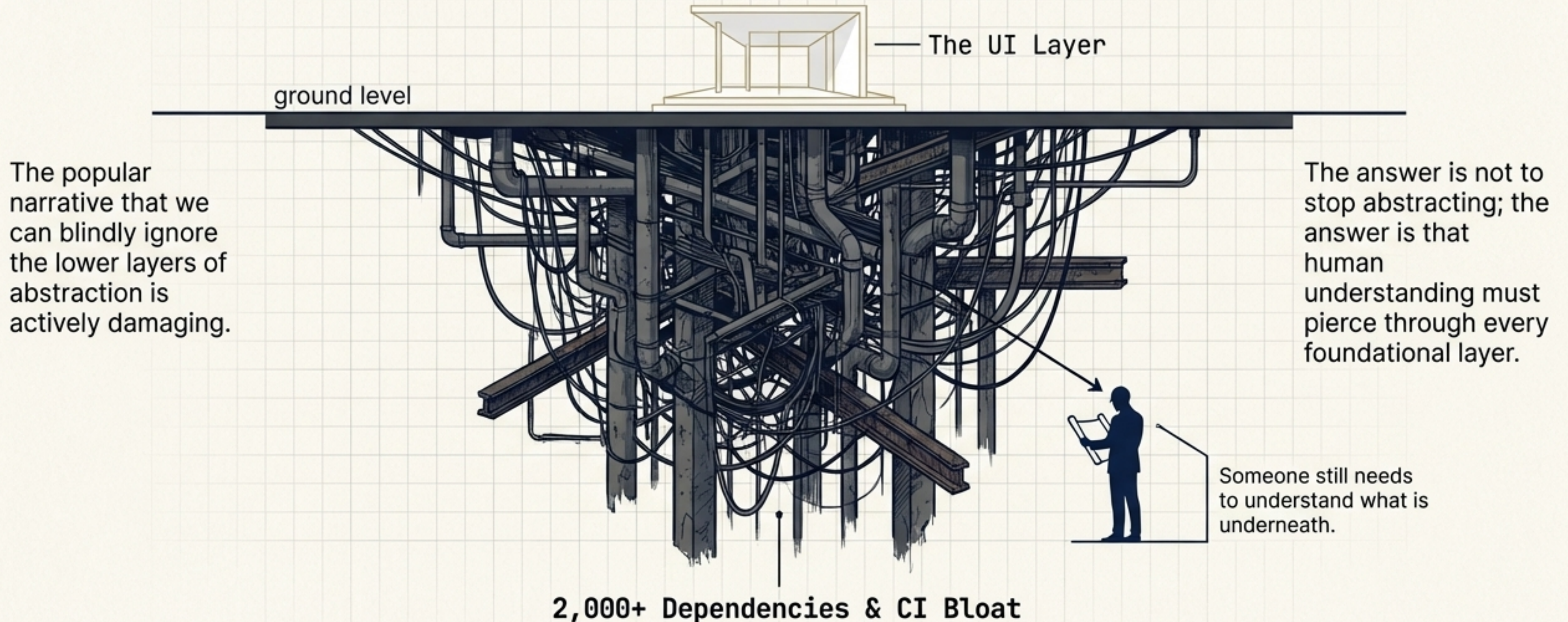
Town Planner to Villager

Planners treat architectural flaws as incidents. They fix the Villager's underlying review process, not just the isolated code snippet.

Each cycle systematically raises the quality floor, mathematically reducing rework for the next phase.

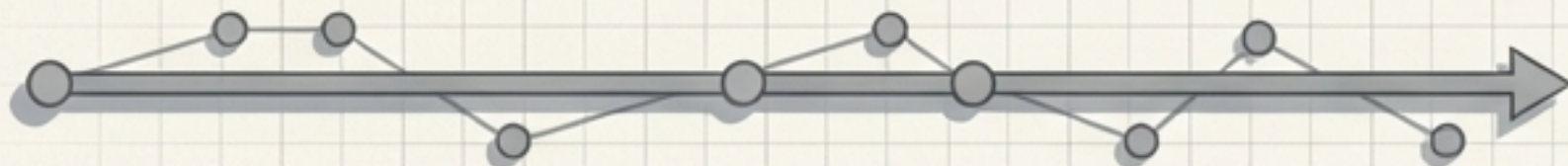
The abstraction trap threatens system fragility.

The Momumt Architectort Extended



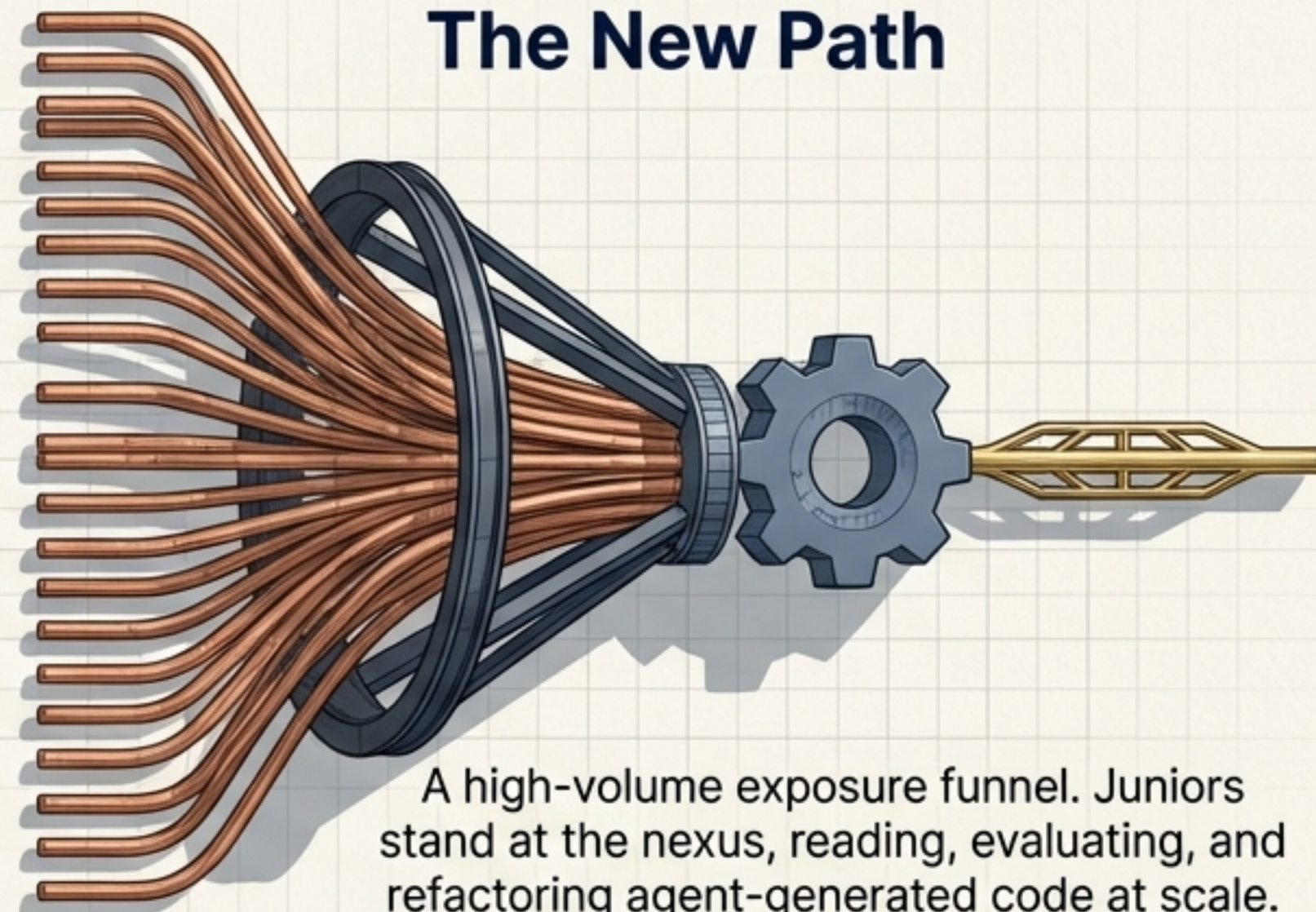
Restructuring the Junior-to-Senior pathway.

The Old Path



A slow, linear timeline of a single developer writing simple code entirely from scratch.

The New Path

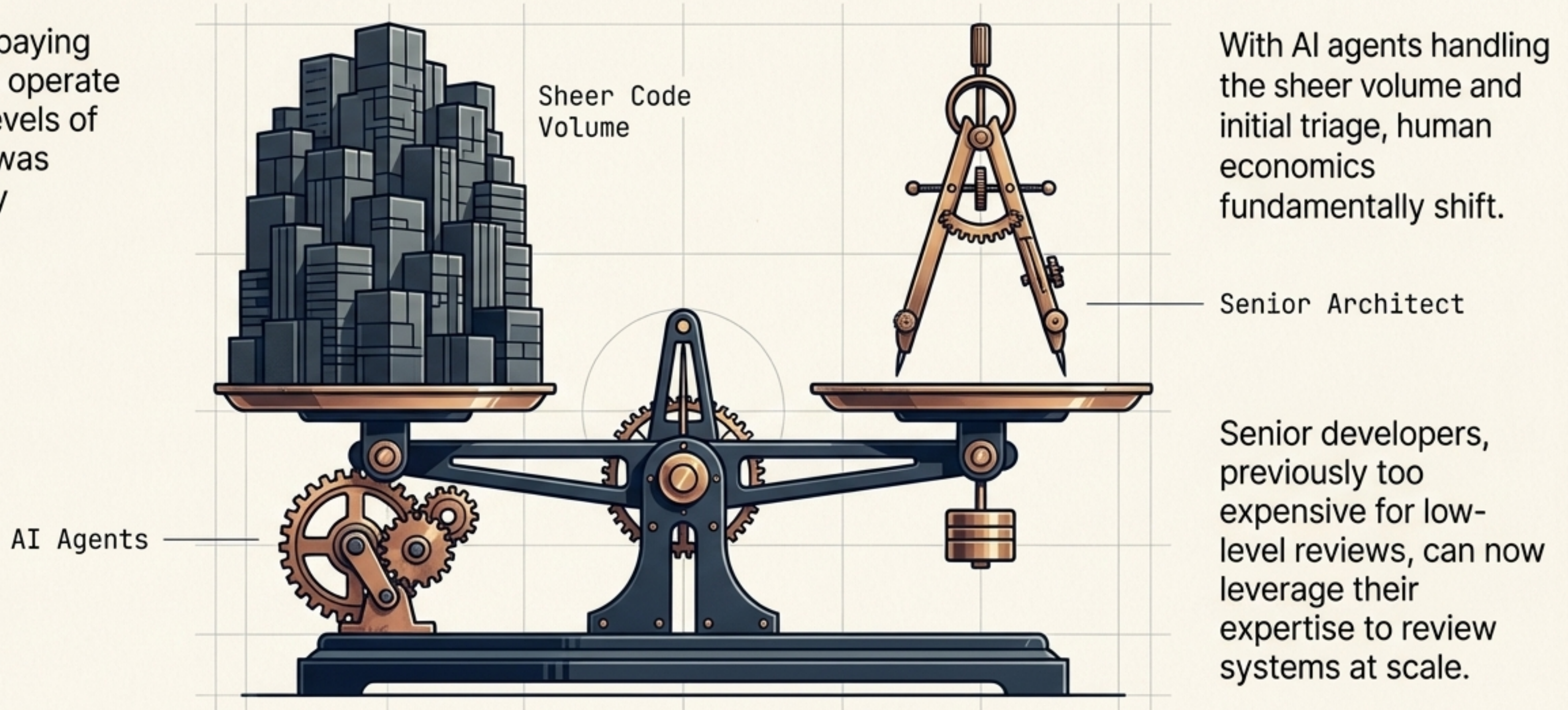


A high-volume exposure funnel. Juniors stand at the nexus, reading, evaluating, and refactoring agent-generated code at scale.

The Result: A vastly higher density of learning per hour due to exposure to exponentially more code patterns and architectures than they could ever write themselves.

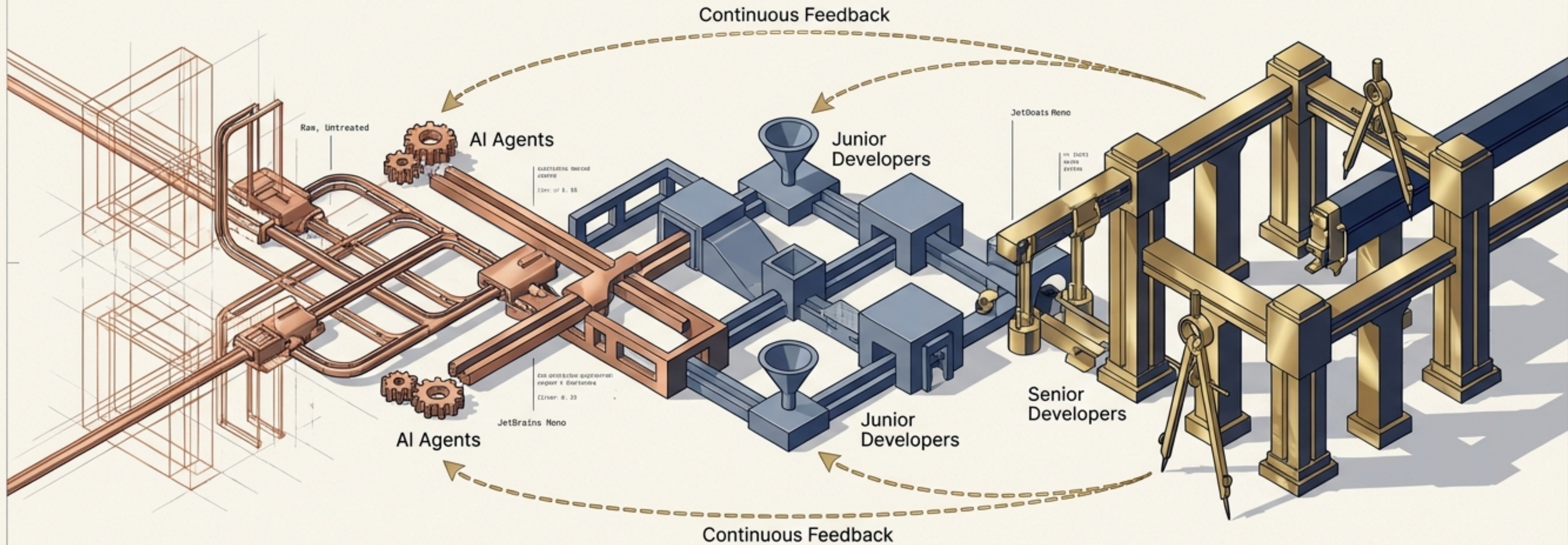
AI makes comprehensive architectural review economically viable.

Historically, paying engineers to operate at multiple levels of abstraction was economically unfeasible.



With AI agents handling the sheer volume and initial triage, human economics fundamentally shift.

A perfectly balanced ecosystem of creation and review.



The Structured Alliance

Agents generate and triage.
Juniors read, understand, and refine.
Seniors deeply debate, verify, and architect.

Continuous Auto-Correction

Strictly separated environments prevent disasters, while backward-flowing guidance guarantees continuous systemic improvement.

The Bottom Line

This is not just a compromise with AI; it is a significantly faster, mathematically superior methodology for building software.